Object Oriented Programming (Java): Model Answers

D Y Patil University, B.Tech Computer Engineering, Semester III. Answers follow the ranking from the previous year paper analysis: topics that appeared in the most papers come first. Each answer gives exam-ready theory and, where asked, a complete Java program with sample output.

Exception Handling (asked in 4 of 4 papers)

What is exception handling?

An exception is an unwanted event during execution that disrupts the normal flow of a program, such as dividing by zero or entering text where a number is expected. Exception handling lets the program catch the problem and continue or end cleanly instead of crashing.
Keywords: **try** (code that may fail), **catch** (handles a specific exception), **finally** (always runs), **throw** (raises an exception manually), **throws** (declares exceptions a method may pass on). All exceptions descend from Throwable, which splits into Error and Exception. Checked exceptions (IOException) must be handled at compile time; unchecked exceptions (ArithmeticException, NumberFormatException) are RuntimeExceptions.

Program: division with NumberFormatException, divide-by-zero and finally

```
import java.util.Scanner;

public class DivideDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter second number: ");
            int b = Integer.parseInt(sc.nextLine());
            int result = a / b;
            System.out.println("Result = " + result);
        } catch (NumberFormatException e) {
            System.out.println("Invalid input: please enter integers
only.");
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " +
e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
```

```
            sc.close();
        }
    }
}
```
Sample output (second number 0): Cannot divide by zero: / by zero, then Finally block executed. With input "abc": Invalid input: please enter integers only, then the finally line.

Multiple catch statements

One try block can have several catch blocks, each handling a different exception type. They are checked in order, so subclasses must come before their superclass (ArithmeticException before Exception) or the code will not compile.

```
public class MultiCatch {
    public static void main(String[] args) {
        try {
            int[] arr = new int[3];
            arr[5] = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic problem");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index problem");
        } catch (Exception e) {
            System.out.println("Some other problem");
        }
    }
}
```
Output: Arithmetic problem (10 / 0 is evaluated first).

Significance of finally; can it exist without try?

The finally block runs whether or not an exception occurs, so it is used for clean-up such as closing files, scanners and database connections. It runs even if the try block has a return statement. It runs every time except when the JVM exits (System.exit) or crashes.
A finally block cannot exist on its own: it must follow a try block (try-catch-finally or try-finally). Writing finally without try is a compile-time error. A try block needs at least one catch or a finally.

OOP Concepts (asked in 4 of 4 papers)

Features of object oriented programming

OOP organises a program as a set of objects that combine data and the methods that work on that data. Main features:

- **Class and object:** a class is a blueprint; an object is an instance of it with its own state.
- **Encapsulation:** binding data and methods in one unit and hiding data using private fields with public getters and setters.
- **Abstraction:** showing only essential details and hiding implementation, done with abstract classes and interfaces.

- **Inheritance:** a child class acquires the properties and methods of a parent class, giving code reuse.
- **Polymorphism:** one name, many forms. Compile-time through method overloading; run-time through method overriding.
- Also: message passing, dynamic binding and data hiding.

Structural (procedural) programming vs OOP

Point	Structural programming	Object oriented programming
Approach	Top-down; program divided into functions	Bottom-up; program divided into objects
Focus	Functions and logic	Data and the objects that own it
Data security	Data is global and easily accessed	Data is hidden using encapsulation
Reuse	Limited, by copying or calling functions	High, through inheritance
Extending	Hard for large programs	Easy to modify and extend
Examples	C, Pascal	Java, C++, Python

OOP in Java vs C++

Feature	Java	C++
Multiple inheritance	Not with classes; achieved with interfaces	Supported directly
Pointers	No explicit pointers	Supported
Memory management	Automatic garbage collection	Manual (new and delete, destructors)
Operator overloading	Not supported	Supported
Platform	Platform independent (bytecode on JVM)	Platform dependent (compiled to machine code)
Structure	Everything is inside a class	Global functions and classes both allowed
Header files	Uses packages and import	Uses header files and #include

Class and object with example

A class defines the fields and methods common to a kind of thing. An object is created from it with the new keyword and has its own copy of the fields.

```
class Student {
    int rollNo;
    String name;

    Student(int r, String n) { rollNo = r; name = n; }

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(1, "Asha");
        Student s2 = new Student(2, "Ravi");
        s1.display();
        s2.display();
    }
}
```
Output: 1 Asha, then 2 Ravi.

Threads (asked in 4 of 4 papers)

Thread and multithreading

A thread is the smallest unit of execution inside a process. Multithreading runs two or more threads at the same time within one program so the CPU is used well and long tasks do not freeze the program. Threads share the same memory, so they are lighter than separate processes. A thread is created either by extending the Thread class or by implementing the Runnable interface; start() begins execution and calls run().

Life cycle of a thread

- **New:** the thread object is created but start() has not been called.
- **Runnable:** start() was called; the thread is ready and waits for CPU time from the scheduler.
- **Running:** the scheduler has picked it and run() is executing.
- **Blocked / Waiting / Timed waiting:** the thread is paused, for example waiting for a lock, for another thread (wait, join) or sleeping (sleep(ms)). It returns to Runnable when the condition ends.
- **Terminated (Dead):** run() has finished or the thread was stopped by an uncaught exception. It cannot be restarted.

Program: thread printing 1 to 10

```
class NumberThread extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(i);
            try { Thread.sleep(300); } catch (InterruptedException e)
{ }
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
```

```
        NumberThread t = new NumberThread();
        t.start();
    }
}
```

Program: two threads, odd numbers 11 to 20 and even numbers 1 to 10

```
class OddThread extends Thread {
    public void run() {
        for (int i = 11; i <= 20; i += 2) {
            System.out.println("Odd: " + i);
            try { Thread.sleep(200); } catch (InterruptedException e)
{ }
        }
    }
}

class EvenThread extends Thread {
    public void run() {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println("Even: " + i);
            try { Thread.sleep(200); } catch (InterruptedException e)
{ }
        }
    }
}

public class TwoThreads {
    public static void main(String[] args) {
        new OddThread().start();
        new EvenThread().start();
    }
}
```

Output: the odd values 11, 13, 15, 17, 19 and even values 2, 4, 6, 8, 10 appear interleaved; the exact order can vary from run to run because the scheduler decides.

Types of Inheritance (asked in 4 of 4 papers)

Inheritance lets a subclass acquire the fields and methods of a superclass using the extends keyword (implements for interfaces). It gives code reuse, easier maintenance and run-time polymorphism. It also lets a programmer add new behaviour without changing existing code.

- **Single:** one subclass extends one superclass (B extends A).
- **Multilevel:** a chain of classes (C extends B, B extends A).
- **Hierarchical:** several subclasses extend the same superclass (B and C extend A).
- **Multiple:** one class inherits from more than one parent. Java does not allow this with classes, to avoid the diamond problem (ambiguity when two parents define the same

method), but allows it through interfaces.

- **Hybrid:** a combination of the above; in Java it is only possible using interfaces.

Example covering single, multilevel and multiple (through interfaces)

```
class Animal {
    void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {              // single
    void bark() { System.out.println("Barking"); }
}
class Puppy extends Dog {               // multilevel
    void weep() { System.out.println("Weeping"); }
}

interface Printable { void print(); }
interface Showable  { void show(); }

class Demo implements Printable, Showable {   // multiple via
interfaces
    public void print() { System.out.println("Print"); }
    public void show()  { System.out.println("Show"); }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat(); p.bark(); p.weep();
        Demo d = new Demo();
        d.print(); d.show();
    }
}
```

Output: Eating, Barking, Weeping, Print, Show.

String and StringBuffer (asked in 4 of 4 papers)

A String is an immutable sequence of characters: once created it cannot be changed, and every modification creates a new object. StringBuffer is mutable and thread-safe: it changes the same object in place, so it is better for repeated modification.

Common String methods

- **length():** number of characters. "Java".length() gives 4.
- **charAt(i):** character at index i. "Java".charAt(2) gives 'v'.
- **substring(b, e):** part from b to e-1. "Hello".substring(1, 4) gives "ell".
- **toUpperCase() / toLowerCase():** changes case.
- **equals() / equalsIgnoreCase():** compares content.

- **indexOf(x) / replace(a, b) / trim() / concat():** search, replace, remove outer spaces, join.

Common StringBuffer methods

- **append(x):** adds to the end.
- **insert(i, x):** inserts at index i.
- **delete(s, e):** removes characters from s to e-1.
- **reverse():** reverses the content.
- **replace(s, e, str):** replaces a range.
- **capacity() / length():** current capacity (default 16) and length.

```
public class StringDemo {
    public static void main(String[] args) {
        String s = "Hello";
        s.concat(" World");                 // s unchanged (immutable)
        System.out.println(s);              // Hello

        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");
        sb.insert(5, ",");
        System.out.println(sb);             // Hello, World
        sb.reverse();
        System.out.println(sb);             // dlroW ,olleH
        sb.delete(0, 6);
        System.out.println(sb);             // ,olleH
    }
}
```

AWT vs Swing, Layouts and GUI (asked in 3 of 4 papers)

Difference between AWT and Swing

AWT	Swing
Heavyweight: uses native operating system components	Lightweight: written fully in Java
Look and feel depends on the platform	Pluggable look and feel, same on all platforms
Fewer components (Button, Label, TextField)	Richer set (JTable, JTree, JTabbedPane, JComboBox)
Package java.awt	Package javax.swing, built on top of AWT
Not MVC based	Follows an MVC-like design
Component names: Frame, Button	Names start with J: JFrame, JButton

Swing is preferred because it is platform independent in appearance, has more and better components, and is faster and more flexible to customise.

Swing class hierarchy

Object, then Component (java.awt), then Container, then JComponent. JComponent is the parent of most Swing widgets (JButton, JLabel, JTextField, JPanel, JTable and others). Top-level containers are separate: Window, then Frame, then JFrame (also JDialog and JApplet). Components are placed in a JFrame's content pane.

Layout managers

- **FlowLayout:** components placed left to right, wrapping to the next row. Default for JPanel.
- **BorderLayout:** five regions: North, South, East, West, Center. Default for JFrame.
- **GridLayout:** equal-sized cells in rows and columns.
- **CardLayout:** stacks components like cards, showing one at a time.
- **BoxLayout:** arranges components in a single row or column.
- **GridBagLayout:** flexible grid where components can span rows and columns.

Program: Swing calculator

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {
    JTextField display = new JTextField();
    double first = 0;
    String op = "";
    boolean startNew = true;

    Calculator() {
        setTitle("Calculator");
        setLayout(new BorderLayout());
        display.setEditable(false);
        display.setHorizontalAlignment(JTextField.RIGHT);
        add(display, BorderLayout.NORTH);

        JPanel p = new JPanel(new GridLayout(4, 4, 5, 5));
        String[] keys = {"7","8","9","/","4","5","6","*",
                         "1","2","3","-","0","C","=","+"};
        for (String k : keys) {
            JButton b = new JButton(k);
            b.addActionListener(this);
            p.add(b);
        }
        add(p, BorderLayout.CENTER);
        setSize(300, 350);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setVisible(true);
    }
```

```
    public void actionPerformed(ActionEvent e) {
        String c = e.getActionCommand();
        if (Character.isDigit(c.charAt(0))) {
            if (startNew) { display.setText(""); startNew = false; }
            display.setText(display.getText() + c);
        } else if (c.equals("C")) {
            display.setText(""); op = ""; first = 0; startNew = true;
        } else if (c.equals("=")) {
            if (op.isEmpty() || display.getText().isEmpty()) return;
            double second = Double.parseDouble(display.getText());
            double r = 0;
            switch (op) {
                case "+": r = first + second; break;
                case "-": r = first - second; break;
                case "*": r = first * second; break;
                case "/":
                    if (second == 0) {
                        display.setText("Error"); op = ""; startNew =
true;
                        return;
                    }
                    r = first / second; break;
            }
            display.setText(String.valueOf(r));
            op = ""; startNew = true;
        } else {
            if (display.getText().isEmpty() ||
display.getText().equals("Error")) return;
            first = Double.parseDouble(display.getText());
            op = c; startNew = true;
        }
    }

    public static void main(String[] args) { new Calculator(); }
}
```

Method Overloading vs Method Overriding (asked in 3 of 4 papers)

Method overloading	**Method overriding**
Same method name, different parameter list, in the same class	Same name and same parameters in a subclass and its superclass
Compile-time (static) polymorphism	Run-time (dynamic) polymorphism
Return type may differ, but return type alone cannot distinguish methods	Return type must be the same (or covariant)
Inheritance not required	Inheritance required

Can be static, private or final	Static, private and final methods cannot be overridden
Access modifier can be anything	Cannot reduce the visibility of the parent method

Method overriding: usage and rules

Overriding lets a subclass give its own implementation of a method already defined in its parent. It is used to achieve run-time polymorphism: the version that runs depends on the actual object, not the reference type. Rules: same name, parameters and return type; the child's access level cannot be stricter; final, static and private methods cannot be overridden; the overriding method cannot throw broader checked exceptions; use @Override so the compiler checks it.

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}
public class OverrideDemo {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();     // Dog barks
    }
}
```

Program: overloaded calculateArea() for circle, rectangle and triangle

```
public class AreaDemo {
    double calculateArea(double r) {                    // circle
        return Math.PI * r * r;
    }
    double calculateArea(double l, double b) {          // rectangle
        return l * b;
    }
    double calculateArea(double b, double h, boolean tri) {   //
triangle
        return 0.5 * b * h;
    }
    public static void main(String[] args) {
        AreaDemo a = new AreaDemo();
        System.out.println("Circle area    = " + a.calculateArea(5));
        System.out.println("Rectangle area = " + a.calculateArea(4,
6));
        System.out.println("Triangle area  = " + a.calculateArea(4, 6,
true));
    }
}
```

Output: Circle area = 78.53981633974483, Rectangle area = 24.0, Triangle area = 12.0. The extra boolean parameter is used only to give the triangle method a different signature from the rectangle method.

Abstract Class and Interface (asked in 3 of 4 papers)

Abstract class	Interface
Declared with abstract; can have abstract and normal methods	Declared with interface; methods are abstract by default (default and static methods allowed since Java 8)
Can have instance variables of any kind	Variables are public static final constants
Can have constructors	No constructors
A class extends only one abstract class	A class can implement many interfaces
Uses extends	Uses implements
For related classes sharing code	For a capability that unrelated classes can share

Program: abstract class Shape with Sphere, Cone and Cylinder

```
abstract class Shape {
    double r, h;
    abstract double area();
    abstract double volume();
}
class Sphere extends Shape {
    Sphere(double r) { this.r = r; }
    double area()   { return 4 * Math.PI * r * r; }
    double volume() { return (4.0 / 3) * Math.PI * r * r * r; }
}
class Cone extends Shape {
    Cone(double r, double h) { this.r = r; this.h = h; }
    double area() {
        double l = Math.sqrt(r * r + h * h);
        return Math.PI * r * (r + l);
    }
    double volume() { return Math.PI * r * r * h / 3; }
}
class Cylinder extends Shape {
    Cylinder(double r, double h) { this.r = r; this.h = h; }
    double area()   { return 2 * Math.PI * r * (r + h); }
    double volume() { return Math.PI * r * r * h; }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] s = { new Sphere(3), new Cone(3, 4), new Cylinder(3,
4) };
        String[] n = { "Sphere", "Cone", "Cylinder" };
        for (int i = 0; i < s.length; i++) {
```

```
            System.out.printf("%s: area = %.2f, volume = %.2f%n",
                              n[i], s[i].area(), s[i].volume());
        }
    }
}
```

Output: Sphere: area = 113.10, volume = 113.10; Cone: area = 75.40, volume = 37.70; Cylinder: area = 131.95, volume = 113.10.

Program: student marks with grace marks (base class and interface)

```
class Student {
    int academicMarks;
    Student(int m) { academicMarks = m; }
    int getAcademic() { return academicMarks; }
}
interface Sports {
    int GRACE = 5;                  // fixed grace marks, same for all
    int getGrace();
}
class Result extends Student implements Sports {
    Result(int m) { super(m); }
    public int getGrace() { return GRACE; }
    int total() { return getAcademic() + getGrace(); }
}
public class GraceDemo {
    public static void main(String[] args) {
        Result r = new Result(72);
        Student s = r;              // base class reference
        Sports sp = r;              // interface reference
        System.out.println("Academic marks = " + s.getAcademic());
        System.out.println("Sports grace   = " + sp.getGrace());
        System.out.println("Total marks    = " + r.total());
    }
}
```

Output: Academic marks = 72, Sports grace = 5, Total marks = 77.

Graphics Programs (asked in 3 of 4 papers)

Display "All the Best" using a frame

```
import java.awt.*;
import java.awt.event.*;

public class AllTheBest extends Frame {
    AllTheBest() {
        setTitle("Greeting");
        setSize(400, 200);
```

```
        setVisible(true);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) { System.exit(0);
}
        });
    }
    public void paint(Graphics g) {
        g.setFont(new Font("Arial", Font.BOLD, 30));
        g.setColor(Color.BLUE);
        g.drawString("All the Best", 100, 110);
    }
    public static void main(String[] args) { new AllTheBest(); }
}
```

Draw a smiling face using Graphics methods

```
import javax.swing.*;
import java.awt.*;

public class Smiley extends JPanel {
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.YELLOW);
        g.fillOval(50, 50, 200, 200);          // face
        g.setColor(Color.BLACK);
        g.drawOval(50, 50, 200, 200);          // outline
        g.fillOval(105, 110, 25, 35);          // left eye
        g.fillOval(170, 110, 25, 35);          // right eye
        g.drawArc(90, 110, 120, 100, 200, 140);   // smile
    }
    public static void main(String[] args) {
        JFrame f = new JFrame("Smiley");
        f.add(new Smiley());
        f.setSize(320, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Recursion Programs (asked in 3 of 4 papers)

Factorial using recursion

```
import java.util.Scanner;

public class Factorial {
    static long fact(int n) {
```

```
        if (n == 0 || n == 1) return 1;     // base case
        return n * fact(n - 1);             // recursive call
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Factorial of " + n + " = " + fact(n));
    }
}
```

Output for 5: Factorial of 5 = 120.

Fibonacci series using recursion

```
public class Fibonacci {
    static int fib(int n) {
        if (n <= 1) return n;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        int terms = 8;
        for (int i = 0; i < terms; i++) System.out.print(fib(i) + "
");
    }
}
```

Output: 0 1 1 2 3 5 8 13.

JVM and Platform Independence (asked in 3 of 4 papers)

Java is platform independent

The Java compiler (javac) does not produce machine code. It converts the source file (.java) into bytecode (.class), which is a neutral instruction set. Any device with a Java Virtual Machine for its operating system can run that same bytecode, so the program is written once and runs anywhere (WORA). The JVM is platform specific, but the bytecode is not.

Working of the JVM

- **Class loader subsystem:** loads .class files into memory (loading), checks them (linking: verification, preparation, resolution) and initialises static data.
- **Runtime data areas:** the method area (class data, static variables), the heap (objects), thread stacks (method calls and local variables), the PC register (next instruction) and native method stacks.
- **Bytecode verifier:** checks that the code is safe and follows Java rules.
- **Execution engine:** runs the bytecode using the interpreter, the Just-In-Time (JIT)

compiler that converts frequently used code to native code for speed, and the garbage collector that frees unused objects.

- **Native method interface:** lets Java call native libraries written in C or C++.

Flow: source code, then javac, then bytecode, then class loader, then verifier, then interpreter and JIT, then output on the machine.

Array Programs (asked in 3 of 4 papers)

Sum and average of array elements with user input

```
import java.util.Scanner;

public class ArraySum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many elements? ");
        int n = sc.nextInt();
        int[] a = new int[n];
        int sum = 0;
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element " + (i + 1) + ": ");
            a[i] = sc.nextInt();
            sum += a[i];
        }
        double avg = (double) sum / n;
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
    }
}
```

Output for 10, 20, 30: Sum = 60, Average = 20.0.

Accept an m x n matrix and display it in natural form

```
import java.util.Scanner;

public class MatrixDisplay {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Rows: ");
        int m = sc.nextInt();
        System.out.print("Columns: ");
        int n = sc.nextInt();
        int[][] a = new int[m][n];
        System.out.println("Enter " + (m * n) + " elements:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();
```

```
        System.out.println("Matrix:");
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++)
                System.out.print(a[i][j] + "\t");
            System.out.println();
        }
    }
}
```

Constructor Overloading (asked in 2 of 4 papers)

A constructor is a special method with the same name as the class and no return type; it runs when an object is created and initialises it. Constructor overloading means a class has more than one constructor with different parameter lists (number, type or order). The compiler picks the one matching the arguments in the new statement.

```
class Box {
    double length, width, height;

    Box() { length = width = height = 1; }                    //
default
    Box(double s) { length = width = height = s; }            // cube
    Box(double l, double w, double h) {                       //
cuboid
        length = l; width = w; height = h;
    }
    double volume() { return length * width * height; }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(3);
        Box b3 = new Box(2, 3, 4);
        System.out.println(b1.volume());   // 1.0
        System.out.println(b2.volume());   // 27.0
        System.out.println(b3.volume());   // 24.0
    }
}
```

Multilevel Inheritance: Volume of a Sphere (asked in 2 of 4 papers)

```
import java.util.Scanner;

class Radius {                          // base class
    double r;
```

```
    void getRadius() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter radius: ");
        r = sc.nextDouble();
    }
}
class Area extends Radius {             // derived from Radius
    double area() { return 4 * Math.PI * r * r; }     // area of the
sphere
}
class Volume extends Area {             // derived from Area
    void showVolume() {
        double v = (4.0 / 3) * Math.PI * r * r * r;
        System.out.println("Area of sphere   = " + area());
        System.out.println("Volume of sphere = " + v);
    }
}
public class SphereDemo {
    public static void main(String[] args) {
        Volume obj = new Volume();
        obj.getRadius();
        obj.showVolume();
    }
}
```

Note: one paper words the middle class as "area of a circle" and another as "area of a sphere". Use pi times r squared if the paper says circle. Output for radius 3: Area = 113.097..., Volume = 113.097....

Access Specifiers (asked in 2 of 4 papers)

Access specifiers (modifiers) control where a class, method or variable can be accessed from.

Specifier	Same class	Same package	Subclass in other package	Other package
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

```
public class Account {
    private double balance;            // only inside this class
    protected String owner;            // package and subclasses
    int branchCode;                    // default: same package
    public double getBalance() { return balance; }   // everywhere
}
```

Argument Passing Mechanism (asked in 2 of 4 papers)

Java always uses pass by value. For primitive types (int, double, char and so on) a copy of the value is passed, so changes inside the method do not affect the caller. For objects and arrays the value passed is a copy of the reference, so the method can change the object's contents (visible to the caller) but cannot make the caller's variable point to a different object.

```
class Num { int value; Num(int v) { value = v; } }

public class ArgDemo {
    static void changePrimitive(int x) { x = 100; }
    static void changeObject(Num n)    { n.value = 100; }
    static void changeArray(int[] a)   { a[0] = 100; }
    static void reassign(Num n)        { n = new Num(999); }

    public static void main(String[] args) {
        int p = 10;
        changePrimitive(p);
        System.out.println("Primitive: " + p);          // 10

        Num obj = new Num(10);
        changeObject(obj);
        System.out.println("Object: " + obj.value);     // 100

        int[] arr = {10, 20};
        changeArray(arr);
        System.out.println("Array: " + arr[0]);         // 100

        Num o2 = new Num(10);
        reassign(o2);
        System.out.println("After reassign: " + o2.value);   // 10
    }
}
```

Output: Primitive: 10, Object: 100, Array: 100, After reassign: 10.

Single-Appearance Questions

Class Rectangle with area and perimeter

```
class Rectangle {
    double length, width;
    Rectangle(double l, double w) { length = l; width = w; }
    double area()      { return length * width; }
    double perimeter() { return 2 * (length + width); }
}
public class RectDemo {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(5, 3);
        System.out.println("Area = " + r.area());            // 15.0
        System.out.println("Perimeter = " + r.perimeter());  // 16.0
```

```
    }
}
```

Sum of first n natural numbers

```
import java.util.Scanner;

public class NaturalSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        int sum = 0;
        for (int i = 1; i <= n; i++) sum += i;
        System.out.println("Sum = " + sum);     // n = 10 gives 55
    }
}
```

nCr and nPr

nPr = n! / (n-r)! and nCr = n! / (r! (n-r)!).

```
import java.util.Scanner;

public class CombPerm {
    static long fact(int n) {
        long f = 1;
        for (int i = 2; i <= n; i++) f *= i;
        return f;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n and r: ");
        int n = sc.nextInt(), r = sc.nextInt();
        long nPr = fact(n) / fact(n - r);
        long nCr = fact(n) / (fact(r) * fact(n - r));
        System.out.println("nPr = " + nPr);     // n=5, r=2 gives 20
        System.out.println("nCr = " + nCr);     // gives 10
    }
}
```

Static data members and methods

A static member belongs to the class, not to any object. There is one copy shared by all objects, created when the class is loaded. A static variable is used for common data such as a counter. A static method can be called with the class name without creating an object, can use only static members directly, and cannot use this or super.

```
class Counter {
    static int count = 0;
```

```
    Counter() { count++; }
    static void show() { System.out.println("Objects created: " +
count); }
}
public class StaticDemo {
    public static void main(String[] args) {
        new Counter(); new Counter(); new Counter();
        Counter.show();      // Objects created: 3
    }
}
```

Packages and their types

A package is a group of related classes and interfaces under one namespace. It avoids name clashes, controls access and makes code easier to organise. It is declared with the package statement as the first line and used with import.

- **Built-in packages:** supplied with Java, such as java.lang (imported automatically), java.util, java.io, java.awt, javax.swing and java.sql.
- **User-defined packages:** created by the programmer.

```
// File: mypack/Hello.java
package mypack;
public class Hello {
    public void show() { System.out.println("Hello from package"); }
}

// File: Test.java (compile with: javac -d . Hello.java)
import mypack.Hello;
public class Test {
    public static void main(String[] args) { new Hello().show(); }
}
```

Type casting and type conversion

Type conversion (widening, automatic): a smaller type is converted to a larger one without loss. Order: byte, short, int, long, float, double. **Type casting (narrowing, manual):** a larger type is converted to a smaller one using a cast, and data may be lost.

```
public class CastDemo {
    public static void main(String[] args) {
        int i = 10;
        double d = i;               // widening: 10.0
        double x = 9.78;
        int y = (int) x;            // narrowing: 9 (fraction lost)
        System.out.println(d + " " + y);
    }
}
```

Check whether a string is a palindrome

```
import java.util.Scanner;

public class Palindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        String rev = new StringBuffer(s).reverse().toString();
        if (s.equalsIgnoreCase(rev))
            System.out.println(s + " is a palindrome");
        else
            System.out.println(s + " is not a palindrome");
    }
}
```

Output: "madam" is a palindrome; "hello" is not a palindrome.